Massey University
College of Sciences

161.304: Comments and Solutions for Computer Laboratory 11

Exercise 1: Easy, easy to start

- 11E1

  If $p = 0.35$, then log-odds are:

  $$\text{logit}(p) = \log\left(\frac{0.35}{1-0.35}\right) \approx -0.62,$$

  since calculating it in R give:

  ```
  log(0.35/(1-0.35))
  ```

- 11E2

  This one is asking for the reverse operation compared to 11E1. Going from log-odds to probability uses the logistic function to accomplish this. Notice the `plogis()` function that is exactly the logistic function (and `qlogis()` would be the logit function):

  ```
  plogis(3.2)
  ```

  And the answer is ≈ 0.96.

- 11E3

  The proportional change in odds is calculated by exponentiating the coefficient:

  ```
  exp(1.7)
  ```

  This means that each unit change in the predictor variable multiplies the odds of the event by approximately 5.5. Make sure you write the log-odds and the odds (hence with the exponential of the linear model $a + 1.7 \times x$) to identify this change. Also remark that the change in probability depends on the value of the probability itself, so on the value of a, the intercept and of that of the predictor x too, since the probability writes:

  $$p(x) = \frac{\exp(a + 1.7 \times x)}{1 + \exp(a + 1.7 \times x)}.$$

- 13E1

  Option (a) will produce more shrinkage, because the prior is more concentrated. If this isn't obvious from the smaller standard deviation, you can always plot the two priors and compare:

  ```
  curve(dnorm(x,0,1), from=-5, to=5, ylab="Linear model scale density")
  curve(dnorm(x,0,2) , add=TRUE , lty=2)
  ```

  Since option (a), shown by the solid density, piles up more mass around zero, it will pull extreme values closer to zero.

  Moreover, I encourage you to have a look at the logistic transformed values to see the effect on the probability:

```
one <- plogis(rnorm(10000, 0, 1))
two <- plogis(rnorm(10000, 0, 2))
dens(one)
dens(two, add = TRUE, lwd = 2)
```

- 13E2

  To turn the model into a multi-level model, we need to assign some hyper-priors. That is priors on parameters of likelihood distribution parameters. In this case, we express $\alpha_{group[i]}$ (a parameter) through a normal prior with another set of parameters: $\overline{\alpha}$ the mean, with a normal prior and σ_α with an Exponential prior. β won't change here as it doesn't depend on the group. We could in theory...

$$\begin{aligned} y_i &\sim \text{Binomial}(1, p_i) \\ \text{logit}(p_i) &= \alpha_{group[i]} + \beta x_i \\ \alpha_{group} &\sim \text{Normal}(\overline{\alpha}, \sigma_\alpha) \\ \overline{\alpha} &\sim \text{Normal}(0, 2) \\ \sigma_\alpha &\sim \text{Exponential}(1) \\ \beta &\sim \text{Normal}(0, 1) \end{aligned}$$

  The numbers we feed into our hyperpriors here are difficult to assess for sensibility since we don't have any data to test the performance of both models and their assumptions.

- 13E3

  It's really the same as the previous model, playing with the intercept α_{group} and these three lines to replace the $\alpha_{group} \sim \text{Normal}(0, 10)$.

$$\begin{aligned} \ldots &\ldots \ldots \\ \alpha_{group} &\sim \text{Normal}(\overline{\alpha}, \sigma_\alpha) \\ \overline{\alpha} &\sim \text{Normal}(0, 2) \\ \sigma_\alpha &\sim \text{Exponential}(1) \\ \ldots &\ldots \ldots \end{aligned}$$

- 13E4

  Again, very similar to both 13E2 and 13E3:

$$\begin{aligned} y_i &\sim \text{Poisson}(\lambda_i) \\ \log(\lambda_i) &= \alpha_{group[i]} + \beta x_i \\ \alpha_{group} &\sim \text{Normal}(\overline{\alpha}, \sigma_\alpha) \\ \overline{\alpha} &\sim \text{Normal}(0, 2) \\ \sigma_\alpha &\sim \text{Exponential}(1) \\ \beta &\sim \text{Normal}(0, 1) \end{aligned}$$

Exercise 2: Quadratic vs MCMC, 11M7

We fit the 2 models:

```
library(rethinking)
data("chimpanzees")
chimpanzees$treatment <- 1 + chimpanzees$prosoc_left + 2*chimpanzees$condition
dat_list <- list(pulled_left = chimpanzees$pulled_left,
   actor = chimpanzees$actor, treatment = chimpanzees$treatment)
q11.4 <- quap(alist(
   pulled_left ~ dbinom(1, p),
   logit(p) <- a[actor] + b[treatment],
```

```
   a[actor] ~ dnorm(0, 1.5),
   b[treatment] ~ dnorm(0, 0.5)
), data = dat_list)
m11.4 <- ulam(alist(
   pulled_left ~ dbinom(1, p),
   logit(p) <- a[actor] + b[treatment],
   a[actor] ~ dnorm(0, 1.5),
   b[treatment] ~ dnorm(0, 0.5)
), data=dat_list, chains=4, log_lik=TRUE)
```

Now if we look quickly at a summary of the posterior, we see that there isn't much difference between the distribution of the parameters, either of the actors or for the treatments. The only parameter that shows a noticeable difference is Actor 2. Looking a little closer, we see that for Actor 2, MCMC results in a posterior shifted slightly to the right. This is because the MCMC posterior is slightly skewed, whereas the quadratic approximation posterior is forced to be Gaussian. Therefore, more density is given to the lower tail and less to the upper tail than in the MCMC posterior.

```
plot(precis(m11.4, depth = 2))
plot(precis(q11.4, depth = 2))
```

Feel free to draw samples from both these models and clearly visualise all distributions, not only a summary via a 89% credibility interval.

Now let's modify our prior distributions. By loosening the prior, we are letting the actor intercepts take even more extreme values. This should have the effect of letting the posterior become even more skewed.

```
q11.4bis <- quap(alist(
   pulled_left ~ dbinom(1, p),
   logit(p) <- a[actor] + b[treatment],
   a[actor] ~ dnorm(0, 10),
   b[treatment] ~ dnorm(0, 0.5)
), data = dat_list)
m11.4bis <- ulam(alist(
   pulled_left ~ dbinom(1, p),
   logit(p) <- a[actor] + b[treatment],
   a[actor] ~ dnorm(0, 10),
   b[treatment] ~ dnorm(0, 0.5)
), data=dat_list, chains=4, log_lik=TRUE)
plot(precis(m11.4bis, depth = 2))
plot(precis(q11.4bis, depth = 2))
```

Yes, this is the case. Let's do the extraction to see the two posterior densities for $\alpha_{Actor[2]}$.

```
q_samp <- extract.samples(q11.4bis)
m_samp <- extract.samples(m11.4bis)
dens(q_samp$a[,2], xlab = "alpha(Actor[2]) (black: quap, red: MCMC)", lwd = 2)
dens(m_samp$a[,2], lwd = 2, col = 2, add = TRUE)
```

We see that the MCMC posterior has much more skew now. However the quadratic approximation posterior is still constrained to be Gaussian. In this case, the quadratic approximation posterior is a pretty bad approximation of the true shape of the posterior.

Exercise 3: The eagle pirate, 11H2

(a) `data(eagles, package = "MASS")`

```
eagle_dat <- data.frame(row.names = rownames((eagles)))
eagle_dat$pirateL <- ifelse(eagles$P == "L", 1, 0)
eagle_dat$victimL <- ifelse(eagles$V == "L", 1, 0)
eagle_dat$pirateA <- ifelse(eagles$A == "A", 1, 0)
eagle_dat$y <- eagles$y
eagle_dat$n <- eagles$n
```

Now two models, the first with `quap()` and the second using `ulam()`. We can compare the parameter estimates using `precis()`. Overall, the estimates are very similar. However, we can note that the MCMC parameters are slightly more extreme than their respective quadratic approximation counterparts. This is because the MCMC posteriors do not have to be strictly Gaussian, and therefore can have a little more skew. If we used wider priors, the difference would be even more evident.

```
eagle_quap <- quap(alist(
   y ~ dbinom(n, p),
   logit(p) <- a + bP * pirateL + bV * victimL + bA * pirateA,
   a ~ dnorm(0, 1.5),
   bP ~ dnorm(0, 1),
   bV ~ dnorm(0, 1),
   bA ~ dnorm(0, 1)
), data = eagle_dat)
eagle_ulam <- ulam(alist(
   y ~ dbinom(n, p),
   logit(p) <- a + bP * pirateL + bV * victimL + bA * pirateA,
   a ~ dnorm(0, 1.5),
   bP ~ dnorm(0, 1),
   bV ~ dnorm(0, 1),
   bA ~ dnorm(0, 1)
), data = eagle_dat, chains=4, log_lik=TRUE, iter = 10000)
plot(precis(eagle_quap))
plot(precis(eagle_ulam))
pairs(eagle_quap)
pairs(eagle_ulam)
```

If you were to use wider priors `dnorm(0,10)` for `a` and `dnorm(0, 5)` for the others, what would happen is that both `bP` and `bV` would be hitting ceiling and floor effects, due to the binomial outcome. The parameter `bP` would be hitting the ceiling, as a large pirate has so big an advantage that many extreme values are all consistent with the data. Basically any value of `bP` that ensures nearly 100% success will do. So there would be a long tail of large values on the right side of the posterior distribution for `bP`. The same thing would be going on with `bV`, but in the opposite direction, hitting the floor.

(b) I will use the `ulam()` estimates, to account for the small asymmetry.

First, the intercept log-odds, `a`, indicates the probability of a successful attempt for a pirate when all of the predictors are at zero. So that means a small immature pirate against a small victim has probability logistic$(0.37) \approx 59\%$ of success.

Second, what about the slope estimates? These estimates just change the intercept for the log-odds. So for example, the probability that a large immature pirate succeeds against a small victim would be logistic$(0.37+2.66) \approx 95.4\%$. That is to say, the large pirate has good chances to succeed, according to the model.

Third, you can generate such predictions for all of the combinations, if you wish. And indeed, that's how the plotting of predictions works.

Before that, what about the proportional odds (odds ratio) interpretations? You can exponentiate each β estimate to get the proportional change in odds that arising from changing the corresponding dummy variable from 0 to 1. Ignoring the confidence intervals for the moment (they will appear in the plots to come), you can quickly exponentiate the β estimates with: `round(exp(coef(eagle_ulam)[2:4]),3)`. So this says that the odds $p/(1-p)$ of a successful

attempt are multiplied by 14.2, when the pirate becomes large. That's a large effect. Similarly, the odds of success are multiplied by 0.06, when the victim becomes large. So if the pirate is large and the victim is large, the proportional change in odds is $14.2 \times 0.06 \approx 0.87$ or about 13% reduction in odds. Finally, the effect of being adult is to improve the odds, multiplying them by about 2.5. These are all relative risk measures. For the absolute measures, it will be nice to plot them.

There are many ways to plot the posterior predictions. I will use a straightforward format with cases on the horizontal and probability/count on the vertical. Here is the code for the first plot, showing proportion success on the vertical axis:

```
eagle_dat$psuccess <- eagle_dat$y / eagle_dat$n
p <- link(eagle_ulam)
yy <- sim(eagle_ulam)
p.mean <- apply(p, 2, mean)
p.PI <- apply(p ,2, PI)
y.mean <- apply(yy ,2, mean)
y.PI <- apply(yy, 2, PI)
# plot raw proportions success for each case
plot(eagle_dat$psuccess, col = rangi2, ylab="successful proportion",
  xlab="case", xaxt="n", xlim=c(0.75,8.25), pch=16)
# label cases on horizontal axis
axis(1, at=1:8, labels=c("LAL","LAS","LIL","LIS","SAL","SAS","SIL","SIS"))
```

And here is the second plot, showing number of successes on the vertical axis:

```
# display posterior predicted probability of success
points(1:8 , p.mean)
for(i in 1:8){lines(c(i,i), p.PI[,i])}
# plot raw counts success for each case
plot(eagle_dat$y , col=rangi2, ylab="successful attempts", xlab="case",
  xaxt="n", xlim=c(0.75,8.25), pch=16)
# label cases on horizontal axis
axis(1 ,at=1:8 ,labels=c("LAL","LAS","LIL","LIS","SAL","SAS","SIL","SIS"))
# display posterior predicted probability of success
points(1:8 ,y.mean)
for(i in 1:8){lines(c(i,i), y.PI[,i])}
```

So what is different between these two plots? The biggest difference is probably that the proportion plot (first) makes the probabilities more comparable, because it ignores the sample size for each case on the horizontal axis. This has the advantage of showing, for example, that SIS attempts are predicted to be somewhat successful, even though only four of them were observed. The count plot (second one, in contrast, makes it hard to see the differing probabilities, because samples size varies so much across cases. On the other hand, the count plot (second one) has the advantage of showing additional uncertainty that arises from the binomial process. Inspect the SIS case again, for example. The observed proportion of SIS successes is outside and below the probability interval (probability plot). However, in the count plot, the model can accommodate the SIS cases, due to additional uncertainty arising from the binomial process. In other words, an additional level of stochasticity is factored into the count plot, and so in total looking also at counts might be necessary to seriously critique the model.

(c) The two models to compare are the one you fitted before, with the three main effects, and the new one, including the interaction $P \times A$. Here is the code to fit the model and compare them:

```
eagle_ulam2 <- ulam(alist(
  y ~ dbinom( n , p ),
  logit(p) <- a + bP*pirateL + bV*victimL + bA*pirateA + bPA*pirateL*pirateA,
  a ~ dnorm(0,1.5),
  bP ~ dnorm(0, 1),
  bV ~ dnorm(0, 1),
  bA ~ dnorm(0, 1),
```

```
  bPA ~ dnorm(0, 1)
), data=eagle_dat, chains = 4, log_lik=TRUE)
compare(eagle_ulam, eagle_ulam2)
```

We see that there is minimal difference, and the non-interaction model is preferred. Thus, we conclude that the interaction term does not have a large impact on the predictive power of the model.

In this case, I feel more secure appealing to actual *biology* to justify the interaction model. There are good behavioural ecological reasons to expect an interaction, and there is good evidence of one. WAIC is not necessarily the most important thing.

Let's look at the estimates for the interaction model:

```
precis(eagle_ulam2)
```

So when the pirate is both large and adult, the log-odds are slightly smaller. This seems like a weird result. Shouldn't being large and adult help? It does. But the main effects for size and age are also increased a wee bit, when an individual is both large and adult. So it's hard to understand what the model is saying, without computing predictions case by case. Do it by replotting the probability and/or the count scale as before :).

Exercise 4: Multi-level frogs

- 13M1

  This corresponds to the multi-level tadpole example in the SR book (starting in section 13.1 on page 415). First, I load the data and prepare the data list as was done in the book and add in the data about predation (binary - yes/no) and size treatment (binary - small/large):

```
data(reedfrogs)
d <- reedfrogs
dat <- list(
  S = d$surv,
  n = d$density,
  tank = 1:nrow(d),
  pred = ifelse(d$pred == "no", 0L, 1L),
  size_ = ifelse(d$size == "small", 1L, 2L))
```

  `ulam()` doesn't like when any data value is called `size`. That's why it's called `size_`. Now, I can define the models. Note that I am running all of them with `log_lik = TRUE` so I can compare them later:

  1. Tank-only model which will serve as our baseline

```
m_Tank <- ulam(alist(
    S ~ dbinom(n, p),
    logit(p) <- a[tank],
    a[tank] ~ dnorm(a_bar, sigma),
    a_bar ~ dnorm(0, 1.5),
    sigma ~ dexp(1)
), data = dat, chains = 4, cores = 4, log_lik = TRUE, iter = 2e4)
```

  2. Predation model

```
m_Pred <- ulam(alist(
    S ~ dbinom(n, p),
    logit(p) <- a[tank] + bp*pred,
    a[tank] ~ dnorm(a_bar, sigma),
    a_bar ~ dnorm(0, 1.5),
    sigma ~ dexp(1),
    bp ~ dnorm(-0.5, 1)
), data = dat, chains = 4, cores = 4, log_lik = TRUE, iter = 2e4)
```

3. Size model

```
m_Size <- ulam(alist(
    S ~ dbinom(n, p),
    logit(p) <- a[tank] + s[size_],
    a[tank] ~ dnorm(a_bar, sigma),
    a_bar ~ dnorm(0, 1.5),
    sigma ~ dexp(1),
    s[size_] ~ dnorm(0, 0.5)
), data = dat, chains = 4, cores = 4, log_lik = TRUE, iter = 2e4)
```

4. Predation + Size model

```
m_Additive <- ulam(alist(
    S ~ dbinom(n, p),
    logit(p) <- a[tank] + bp*pred + s[size_],
    a[tank] ~ dnorm(a_bar, sigma),
    a_bar ~ dnorm(0, 1.5),
    sigma ~ dexp(1),
    bp ~ dnorm(-0.5, 1),
    s[size_] ~ dnorm(0, 0.5)
), data = dat, chains = 4, cores = 4, log_lik = TRUE, iter = 2e4)
```

5. Predation-Size-Interaction model

```
m_Interaction <- ulam(alist(
    S ~ dbinom(n, p),
    logit(p) <- a[tank] + bp*pred + s[size_] +  bps*pred*s[size_],
       # interaction comes in via bP[size_]
    a[tank] ~ dnorm(a_bar, sigma),
    a_bar ~ dnorm(0, 1.5),
    sigma ~ dexp(1),
    c(bp, bps) ~ dnorm(-0.5, 1),
    s[size_] ~ dnorm(0, 0.5)
), data = dat, chains = 4, cores = 4, log_lik = TRUE, iter = 2e4)
```

Wow, now that we have all the models ready, we can assess the variation among tanks. This information is contained within the sigma parameter in all of the models:

```
plot(coeftab(m_Tank, m_Pred, m_Size, m_Additive, m_Interaction),
  pars="sigma", labels=c("Tank", "Predation", "Size", "Additive", "Interaction")
)
```

Quite evidently, omitting `pred` (predation) from our models assigns a lot of variation to the tank variable. Conclusively, we can say that predation explains a lot of the variation across tanks and helps to explain it. Omitting predation from our models simply assigns this variation to the tank intercepts without explaining it. Another way to say it is to notice that `size_` reduces the variation much less than does `pred`. The predictor `size_`, in these models, doesn't help prediction very much, so accounting for it has minimal impact on the estimated variation across tanks. Predation does seem to matter, even if it doesn't explain much of the total variation.

Note that adding a predictor always decreased the posterior mean variation across tanks. Why? Because the predictors are, well, predicting variation. This leaves less variation for the varying intercepts to mop up. In theory, if we had in the form of predictor variables all of the relevant information that determined the survival outcomes, there would be zero variation across tanks.

- 13M2

  Straightforward from a command perspective

```
compare(m_Tank, m_Pred, m_Size, m_Additive, m_Interaction)
```

All of our models are expected to perform similarly in out-of-sample predictions. So how do the posterior samples look like? We don't see all the posterior and the following plots do not tell us how accurate the models are when predicting data. However, they do tell us loads about what the models use to make their predictions:

```
par(mfrow = c(5,1))
plot(precis(m_Tank), main = "m_Tank")
plot(precis(m_Pred), main = "m_Pred")
plot(precis(m_Size, pars = c("a_bar", "sigma", "s"), depth = 2),
  main = "m_Size")
plot(precis(m_Additive, pars = c("a_bar", "sigma", "bp", "s"), depth = 2),
  main = "m_Additive")
plot(precis(m_Interaction, pars = c("a_bar", "sigma", "bp", "bps", "s"),
  depth = 2), main = "m_Interaction")
par(mfrow = c(1,1))
```

For now, I will focus on the posterior distributions of our predation parameter (`bp`) and size parameter (`s`). When inspecting these, it is apparent that the parameter estimates of `bp` are much further from 0 than those for `s`. This holds true across all models. In addition, anytime `bp` is contained in a model, `sigma` (the variation in tank intercepts) decreases drastically.

This is consistent with the model rankings. All models that contain `pred` come first. Even the tank-only model does already very well, because there exists variation among tanks for a multitude of reasons. Prediction and inference of causality are not the same thing, after all.

We could also look at the differences in predictions, e.g. using `m_Tank` (the one from the chapter) and the "best" one `m_Pred` graphically. We first reproduce the figure in the chapter:

```
post1 <- extract.samples(m_Tank)
post2 <- extract.samples(m_Pred)
d$propsurv.est1 <- logistic(apply(post1$a, 2, mean))
d$propsurv.est2 <- logistic(apply(post2$a, 2, mean) + mean(post2$bp)*dat$pred)
plot(d$propsurv , ylim=c(0,1) , pch=16 , xaxt="n" ,
      xlab="tank" , ylab="proportion survival" , col=rangi2 )
axis(1, at=c(1,16,32,48), labels=c(1,16,32,48))
points(d$propsurv.est1)
abline(h=mean(inv_logit(post1$a_bar)), lty=2)
```

And now the same figure but with the predictions made with the model accounting for predation:

```
plot(d$propsurv , ylim=c(0,1) , pch=16 , xaxt="n" ,
      xlab="tank" , ylab="proportion survival" , col=rangi2 )
axis(1, at=c(1,16,32,48), labels=c(1,16,32,48))
points(d$propsurv.est2)
abline(h=mean(inv_logit(post2$a_bar)), lty=2) #2 bars because 2 levels of pred
abline(h=mean(inv_logit(post2$a_bar+mean(post2$bp))), lty=2)
```

- 13M3

  This is a slightly modified version of the `m_Tank` model above. Because the Cauchy distribution comes with very long tails, we run into a few issues of divergent transitions (50% without this control, 25% with it!?!) with default parameters and so we add `control=list(adapt_delta=0.99)` to the call to `ulam()` for more measured sampling of the posterior space:

```
m_TankCauchy <- ulam(alist(
    S ~ dbinom(n, p),
    logit(p) <- a[tank],
    a[tank] ~ dcauchy(a_bar, scale),
    a_bar ~ dnorm(0, 1.5), #but no a mean!
    scale ~ dexp(1)
  ), data = dat, chains = 4, cores = 4, log_lik = TRUE,
  iter = 2e4, control = list(adapt_delta = 0.99))
```

Now let's compare the posterior means of `m_Tank` and the new `m_TankCauchy` for their estimates of the α_{tank} parameters:

```
a_Tank <- apply(extract.samples(m_Tank)$a, 2, mean)
a_TankCauchy <- apply(extract.samples(m_TankCauchy)$a, 2, mean)
plot(a_Tank, a_TankCauchy, pch = 16, col = rangi2,
  xlab = "intercept (Gaussian prior)", ylab = " intercept (Cauchy prior)"
)
abline(a = 0, b = 1, lty = 2)
```

The dashed line show the values for which the intercepts are equal in the two models. For most of our intercepts (α_{tank}), both the Cauchy-prior model and the Gaussian-prior model are basically creating the same results (i.e. points on the dashed line). However, once we hit extreme α_{tank} under the Gaussian prior, the α_{tank} estimates of the Cauchy prior are even more extreme by comparison. This is because of how much adaptive shrinkage is going on. In the tanks on the right-hand side of the plot above, extreme proportions of tadpoles survived the experiment. These estimates are shrunk towards the population (i.e. all tanks) mean. Since the Gaussian distribution is more concentrated than the Cauchy distribution, the Gaussian estimates have more shrinkage applied to them and so fall to lower values.

- 13M4

  Well again another version of either `m_Tank` or `m_TankCauchy`

```
m_TankStudent <- ulam(alist(
    S ~ dbinom(n, p),
    logit(p) <- a[tank],
    a[tank] ~ dstudent(2, a_bar, scale),
    a_bar ~ dnorm(0, 1.5), #but no a mean!
    scale ~ dexp(1)
  ), data = dat, chains = 4, cores = 4, log_lik = TRUE,
  iter = 2e4)
```

And we do a similar analysis looking at the consequence for the α_{tank}:

```
a_TankStudent <- apply(extract.samples(m_TankStudent)$a, 2, mean)
plot(a_Tank, a_TankStudent, pch = 16, col = rangi2,
  xlab = "intercept (Gaussian prior)", ylab = " intercept (Student prior)"
)
abline(a = 0, b = 1, lty = 2)
plot(a_TankStudent, a_TankCauchy, pch = 16, col = rangi2,
  xlab = "intercept (Student prior)", ylab = " intercept (Cauchy prior)"
)
abline(a = 0, b = 1, lty = 2)
```

And we see the same sort of effect, but to a lesser extent. The Gaussian prior causes the more shrinkage of the extreme values than the Student and Cauchy priors do. The Student prior causes more shrinkage of the extreme values than the Cauchy prior does. That is what accounts for those 5 extreme points on the right of the plots above. Notice that the righthand side of the plots correspond to situations, when all tadpoles survived.

- 13H4

  Haven't had time to have a look at this one in details yet. But I believe we have tackled most of this in 13M1?!

Exercise 5: Number of kids in Bengal, 13H1

```
data(bangladesh)
```

```
d <- bangladesh
sort(unique(d$district))
d$district_id <- as.integer(as.factor(d$district))
sort(unique(d$district_id))
```

First, we prepare the data into a list.

```
dat_list <- list(
  C = d$use.contraception,
  D = d$district_id
)
```

Not for the model fitting:

1. Fixed-effect

   ```
   m_Fixed <- ulam(alist(
     C ~ bernoulli(p), # this is the same as dbinom(1, p)
     logit(p) <- a[D],
     a[D] ~ dnorm(0, 1.5)
   ), data = dat_list, chains = 4, cores = 4, log_lik = TRUE)
   ```

2. Varying-intercept

   ```
   m_Varying <- ulam(alist(
     C ~ dbinom(1, p),
     logit(p) <- a[D],
     a[D] ~ normal(a_bar, sigma),
     a_bar ~ normal(0, 1.5),
     sigma ~ exponential(1)
   ), data = dat_list, chains = 4, cores = 4, log_lik = TRUE)
   ```

Now to make our predictions:

```
# compute posterior means
p_Fixed <- apply(inv_logit(extract.samples(m_Fixed)$a), 2, mean)
p_Varying <- apply(inv_logit(extract.samples(m_Varying)$a), 2, mean)
# compute raw estimate from data in each district
tab <- table(d$use.contraception, d$district_id) # contraception no/yes per district
n_per_district <- colSums(tab) # number of observations per district
p_raw <- as.numeric(tab[2, ] / n_per_district) # raw proportion per district
nd <- max(dat_list$D) # number of districts
```

Next, we plot expected proportions using contraception as a function of district ID

```
plot(NULL, xlim = c(1, nd), ylim = c(0, 1), ylab = "prob use contraception",
  xlab = "district")
points(1:nd, p_Fixed, pch = 16, col = rangi2, cex = 3)
points(1:nd, p_Varying, cex = 3)
points(1:nd, p_raw, pch = 3, cex = 3)
abline(h = mean(inv_logit(extract.samples(m_Varying)$a_bar)), # population mean
  lty = 2)
```

As expected, the varying-intercept-estimates (open circles) are shrunk towards the population mean (dashed line), when compared to the fixed-intercept-estimates (blue circles) and the raw proportion (cross symbols). Some are shrunk more than others. Those which are shrunk more have been shrunk because:

- Their sample sizes were small and/or
- Their raw proportions were far from the population mean

Shrinkage is also introduced due to large variation in the values within each clustering variable, which is much easier to demonstrate with continuous observations rather than a binary outcome (contraception used: yes/no).